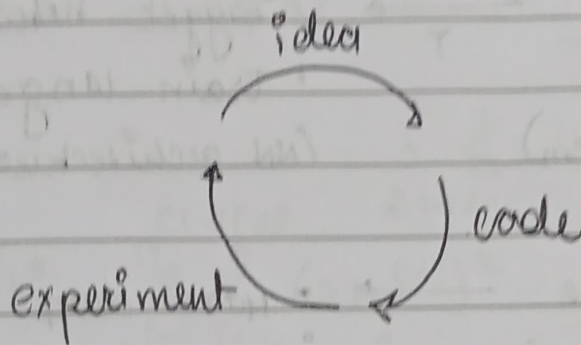
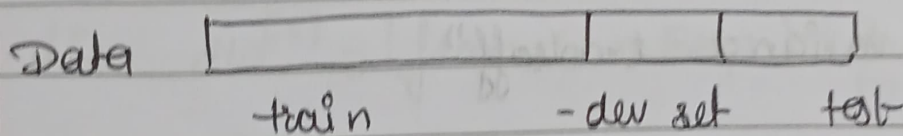


COURSE 2,



Train | dev | test sets



- Hold out
- cross validation.

Bias | Variance.

↙ how our algo is biased to data
 ↘ how it fits the data

Basic Recipe for ML

High bias? $\xrightarrow{\text{Y}}$ • Bigger network
 (training data problem) • Train longer
 (NN architecture search)

High variance $\xrightarrow{\text{Y}}$ • More data
 (dev set problem) • Regularization
 (NN architecture search)

"Bias-variance tradeoff"

REGULARIZATION

$$\min_{w, b} J(w, b) \quad w \in \mathbb{R}^{n \times n}, b \in \mathbb{R}$$

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|w\|_2^2$$

$$L_2 \text{ regularization } \|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w$$

$$L_1 \text{ regularization } \frac{\lambda}{2m} \sum_{i=1}^m |w| = \frac{\lambda}{2m} \|w\|_1$$

λ = regularization parameter

Neural Networks:

$$J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|w^{[l]}\|_2^2$$

$$\|w^{[L]}\|_2^2 = \sum_{i=1}^n \sum_{j=1}^n (w_{ij}^{[L]})^2, \quad w = (w^{[L-1]}, w^{[L]})$$

"Frobenius norm" $\| \cdot \|_F^2$

$$dw \quad dw^{[L]} = (\text{from backprop}) + \frac{\lambda}{m} w^{[L]}$$

$$w^{[L]} = w^{[L]} - \alpha dw^{[L]}$$

$$\frac{\partial J}{\partial w^{[L]}} = dw^{[L]}$$

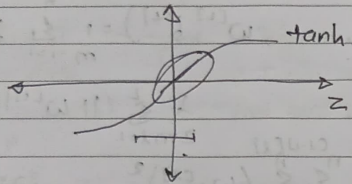
It is also called weight decay.

$$w := w^{[L]} - \alpha \left[\frac{\lambda}{m} w^{[L]} + (\text{from bp}) \right]$$

$$w := w^{[L]} - \alpha \underbrace{\frac{\lambda}{m} w^{[L]}}_{\text{weight is reducing, hence weight decay}} - \alpha (\text{from bp})$$

$\rightarrow$ weight is reducing, hence weight decay.

how regularization help with overfitting.



in smaller parameters, our function's linear.

i.e. every layer $\approx$ linear.

$\therefore$ it cannot fit complex sets.

Dropout regularization:

go through each layer & set some probability of eliminating a node in your network.

Implementing dropout

i) Inverted dropout:

if $l=3$.

$d_3 = \text{np.random.rand}(a_3.\text{shape}[0], a_3.\text{shape}[1])$

d_3 matrix.

$a_3 = \text{np.multiply}(a_3, d_3)$

a_3 $\neq$ keep prob.

$\rightarrow$ element wise multiplication - Hom.

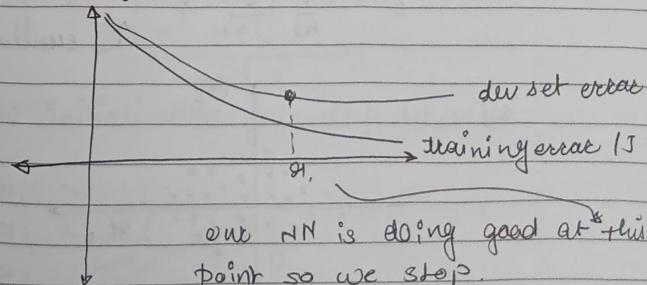
$$z^{[4]} = w^{[4]} a^{[3]} + b^{[4]}$$

ensures that the expected value shut off. $\rightarrow$ since 20% of nodes are randomly

does not change. The value of $z^{[4]}$ will go down, to not change the expected value, we need to divide $a^{[3]}$ by 0.8 (0.8 was our keep prob).

Data Augmentation $\rightarrow$ giving same image in diff. ways.

early stopping:

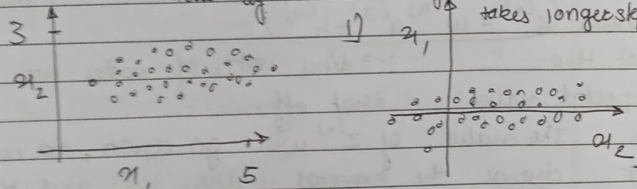


our NN is doing good at this point so we stop.

keep prob.

- but you can't optimize cost funⁿ J.
- not overfit

Normalizing outputs: [to make the cost funⁿ more symm. so the gradient descent takes longer steps]



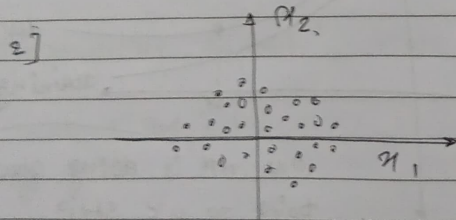
1) subtract mean: $\mu = \frac{1}{m} \sum_{i=1}^m x_i$

$$x_i' = x_i - \mu$$

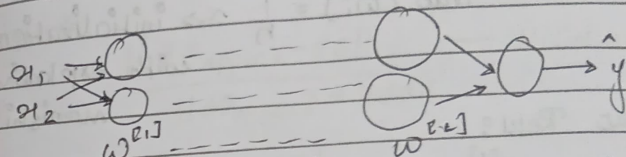
2) normalize variance.

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m x_i'^2 \neq 2$$

↑ element-wise.



vanishing / exploding gradients:



$$g(z) = z \text{ (linear)}$$

$$b^{[1]} = 0$$

becomes $\hat{y}$

$$y = w^{[1]} w^{[2]} \dots w^{[L-1]} x$$

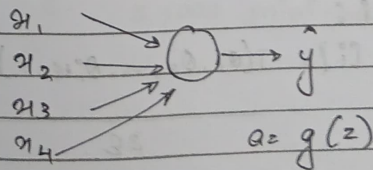
explodes.

$$\hat{y} = w^{[1]} \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}^{L-1} x$$

vanishes.

$$\hat{y} = w^{[1]} \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}^{L-1} x$$

weight initialization for deep network.



$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n \nearrow 0$$

large $n \rightarrow$ smaller w_i

$\text{var}(w_i) = \frac{1}{n} \rightarrow$ initialization helps
with exploding/
vanishing

for Relu:

$$w^{[1]} = \text{np.random.randn}(\text{slope}) * \text{np.sqrt}\left(\frac{2}{n^{[1-1]}}\right)$$

$\rightarrow$ works better.

other activations:

for tanh

$$\sqrt{\frac{1}{n^{[1-1]}}} \quad \text{[Xavier initialization on]}$$

Relu

$$\sqrt{\frac{2}{n^{[1-1]} + n^{[1]}}}$$

Numerical approximation of gradients:

Gradient checking:

- 1] Take $w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}$ and reshape into a big vector θ .
- 2] Same for $dw^{[1]}, db^{[1]}, \dots, dw^{[L]}, db^{[L]}$ into $d\theta$.

for each i :

$$d\text{approx}[i] = J(\theta_1, \theta_2, \dots, \theta_i + \epsilon, \dots) - J(\theta_1, \theta_2, \dots, \theta_i - \epsilon, \dots)$$

22.

$$\approx d\theta[i] = \frac{\partial J}{\partial \theta_i}$$

check

$$\frac{\|d\theta_{\text{approx}} - d\theta\|_2}{\|d\theta_{\text{approx}}\| + \|d\theta\|} \quad \text{--- euclidean dist.}$$

if this is $\approx 10^{-7} \rightarrow$ great!
 $10^{-6} \rightarrow$ bug maybe
 $10^{-3} \rightarrow$ nope x.

Mini Batch gradient descent:

split your training set into mini-batches

$$X(n_x, m) = \begin{bmatrix} x^{(1)} & x^{(2)} & x^{(3)} & \dots & x^{(1000)} & \dots & x^{(m)} \end{bmatrix}$$

$\underbrace{\hspace{10em}}_{X \in \mathbb{R}^{13}} \quad \underbrace{\hspace{10em}}_{X \in \mathbb{R}^{13}} \quad \underbrace{\hspace{10em}}_{X \in \mathbb{R}^{13}}$

$$Y(n_x, m) = \begin{bmatrix} y^{(1)} & y^{(2)} & y^{(3)} & \dots & y^{(1000)} & \dots & y^{(m)} \end{bmatrix}$$

$\underbrace{\hspace{10em}}_{Y \in \mathbb{R}^{13}} \quad \underbrace{\hspace{10em}}_{Y \in \mathbb{R}^{13}}$

if $m = 5,000,000$?
5,000 mini-batches of 1,000 each.

for $t = 1, \dots, 5000$

inside for loop,

1 step of gradient descent using $x^{(t)}$, $y^{(t)}$ (as if $m=1000$).

forward prop as $x^{(t)}$

$$z^{[1]} = w^{[1]} x^{(t)} + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

$$a^{[1]} = g(z^{[1]})$$

vectorized implem-
entation.

compute cost $J^{(t)} = \frac{1}{1000} \sum_{i=1}^{1000} L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2000} \sum_{i=1}^{2000} \|w^{[1]}\|_2^2$

backprop to compute gradient cost $J^{(t)}$ (using $x^{(t)}, y^{(t)}$)

$$w^{[1]} := w^{[1]} - \alpha dw^{[1]}, \quad b^{[1]} := b^{[1]} - \alpha db^{[1]}$$

exponentially weighted averages:

in the data: $\{v_0, v_1, v_2, \dots\}$

$$v_t = \beta v_{t-1} + (1-\beta) \theta_t \rightarrow t\text{'s temp}$$

$$\beta = 0.9 \rightarrow \text{10 days temp.}$$

temp of day
before t .

v_t as approx. avg. of $\frac{1}{1-\beta}$ days temp.

if,

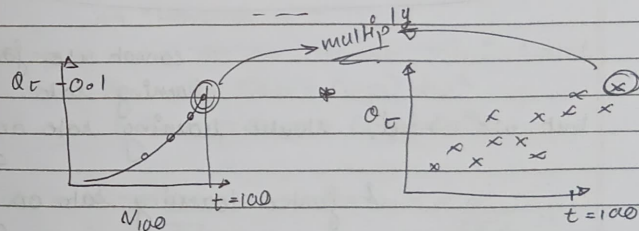
$$v_{100} = 0.9 v_{99} + 0.1 \theta_{100}$$

$$v_{99} = 0.9 v_{98} + 0.1 \theta_{99}$$

$$v_{98} = 0.9 v_{97} + 0.1 \theta_{98}$$

$$\rightarrow v_{100} = 0.1 \theta_{100} + 0.9 v_{99} (0.1 \theta_{99} + 0.9 v_{98} \dots)$$

$$v_{100} = 0.1 \theta_{100} + 0.1 \times 0.9 \theta_{99} + 0.1 \times (0.9)^2 \theta_{98} + \dots$$



Bias correction :-

$$v_t = \beta v_{t-1} + (1-\beta) \theta_t$$

$$v_0 = 0$$

$$v_1 = 0.98 v_0 + 0.02 \theta_1$$

$$v_2 = 0.98 v_1 + 0.02 \theta_2$$

$$\approx 0.98 \times 0.02 \theta_1 + 0.02 \theta_2$$

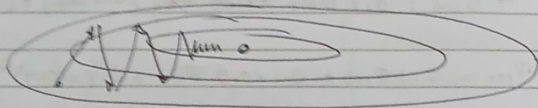
for small values:-

$$\frac{v_t}{1 - \beta^t}$$

when $t = 2$: $1 - \beta^t = 1 - (0.98)^2 = 0.0396$

$$\frac{v_t}{0.0396} = \frac{0.019601 + 0.02 \times 0}{0.0396}$$

gradient descent with momentum.



cannot use faster learning rate.

but we need, slower learning rate on vertical axis

& faster learning rate on horizontal axis.

Momentum:

on iteration t :

compute dw, db on current mini-batch

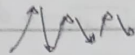
$$v_{dw} = \beta v_{dw} + (1 - \beta) dw$$

$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$w := w - \alpha v_{dw}$$

$$b := b - \alpha v_{db}$$

in the gradient descent:



the vertical direction averages out to almost 0.
∴ damps oscillations

in horizontal direction, since momentum is in same direction average doesn't slow down the movement.

RMS Prop:

on iteration t :

compute dw, db on current mini-batch

$$s_{dw} = \beta s_{dw} + (1 - \beta) dw^2$$

$$s_{db} = \beta s_{db} + (1 - \beta) db^2$$

$$w := w - \alpha \frac{dw}{\sqrt{s_{dw}} + \epsilon}$$

$$b := b - \alpha \frac{db}{\sqrt{s_{db}} + \epsilon}$$

Since,

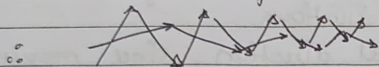
→ Slope large in vertical & small in horizontal

∴ $dw \uparrow$ & $db \downarrow$ $dw \downarrow$ & $db \uparrow$
 $dw \uparrow \uparrow$ and $db \downarrow \downarrow$ $dw \uparrow \downarrow$ & $db \uparrow \uparrow$

in $w := w - \alpha \frac{dw}{\sqrt{s_{dw}}}$

in $b := b - \alpha \frac{db}{\sqrt{S_{db}}} \rightarrow$ dividing by large no.

$\therefore$ oscillations damp out.



Adam optimization algorithm:

(Combine momentum & RMS Prop)

$$v_{dw} = 0, S_{dw} = 0, v_{db} = 0, S_{db} = 0$$

on iteration t :

compute dw, db using current mini-batch

$$v_{dw} = \beta_1 v_{dw} + (1 - \beta_1) dw, \quad \left. \begin{array}{l} \\ \end{array} \right\} \text{momentum}$$

$$v_{db} = \beta_1 v_{db} + (1 - \beta_1) db$$

$$S_{dw} = \beta_2 S_{dw} + (1 - \beta_2) dw^2, \quad \left. \begin{array}{l} \\ \end{array} \right\} \text{RMS prop}$$

$$S_{db} = \beta_2 S_{db} + (1 - \beta_2) db^2$$

$$v_{dw}^{\text{corrected}} = v_{dw} / (1 - \beta_1^t), \quad v_{db}^{\text{corrected}} = v_{db} / (1 - \beta_1^t)$$

$$S_{dw}^{\text{corrected}} = S_{dw} / (1 - \beta_2^t), \quad S_{db}^{\text{corrected}} = S_{db} / (1 - \beta_2^t)$$

$$w = w - \alpha \frac{v_{dw}^{\text{corrected}}}{\sqrt{S_{dw}^{\text{corrected}} + \epsilon}}$$

$$b = b - \alpha \frac{v_{db}^{\text{corrected}}}{\sqrt{S_{db}^{\text{corrected}} + \epsilon}}$$

Hyperparameters choice:

α : needs to be tuned

$$\beta_1 = 0.9 \quad (dw)$$

$$\beta_2 = 0.999 \quad (dw^2)$$

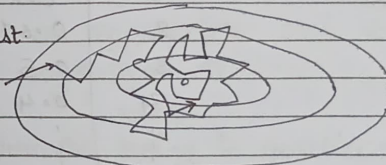
$$\epsilon = 10^{-8}$$

ADAM: adaptive moment estimation

Learning rate Decay:

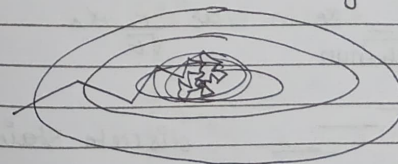
for const.

α :



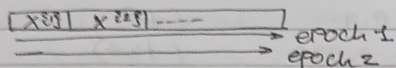
keeps oscillating around.

if we slowly reduce α , it takes slower steps and hence it oscillates in a much tighter region.



Implement:

1 epoch = 1 pass through data.



$$\alpha = \frac{1}{1 + \text{decay rate} \times \text{epoch num}} \alpha_0$$

$$\alpha_0 = 0.2$$

$$\text{decay rate} = 1$$

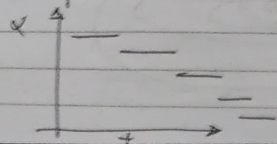
epoch	α
1	0.1
2	0.07
3	0.05
4	0.04
⋮	⋮

exponential decay:

$$\alpha = 0.95^{\text{epoch-num}} \cdot \alpha_0$$

$$\alpha = \frac{k}{\sqrt{\text{epoch-num}}} \alpha_0$$

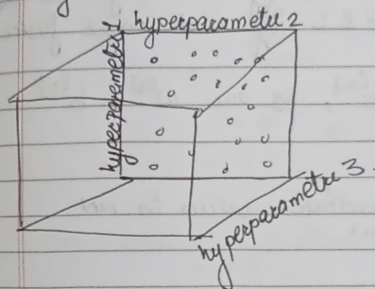
$$\text{at } \frac{k}{\sqrt{t}} \alpha_0$$



discrete staircase.

Tuning Process:

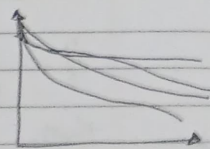
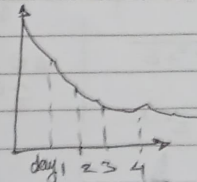
try random values:



using an appropriate scale to pick hyperparameters.

Hyperparameter tuning in practice: Pandas vs. Google

babyst one model, everyday we nudge our parameters a little and observe
train multiple models at the same time.



Normalizing activation in network

we normalize features of one layer so as to train the w & b of next layer faster.

we normalize $z^{[l]}$, so our $w^{[l+1]}$ & $b^{[l+1]}$ train faster.

Given some intermediate values in NN

$$\underbrace{z^{(1)}, \dots, z^{(m)}}_{Z^{(1)}}$$

$$y = \frac{1}{m} \sum_i z^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_i (z_i - y)^2$$

$$z_{\text{norm}}^{(i)} = \frac{z^{(i)} - y}{\sqrt{\sigma^2 + \epsilon}}$$

$$\tilde{z}^{(i)} = y + z_{\text{norm}}^{(i)} + \beta$$

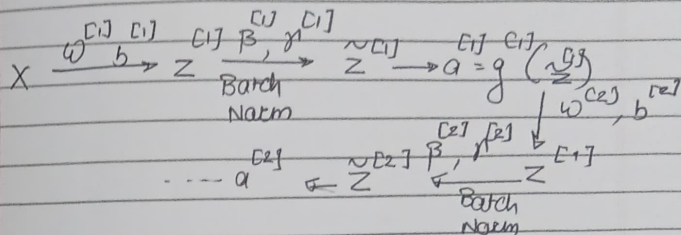
learnable parameters of model.

if, $y = \sqrt{\sigma^2 + \epsilon}$
 $\beta = y$

then, $\tilde{z}^{(i)} = z^{(i)}$

use $\tilde{z}^{(i)}$ instead of $z^{(i)}$

Adding Batch Norm to a Network



Implementing gradient descent

for $t = 1$ to num mini batches

compute forward Prop on $X^{[t]}$

In each hidden layer, use BN to replace $z^{[l]}$ to $\tilde{z}^{[l]}$

use back prop to compute $dw^{[l]}$, $db^{[l]}$, $d\beta^{[l]}$, $d\gamma^{[l]}$

update parameters $w^{[l]} := w^{[l]} - \alpha dw^{[l]}$

$\beta^{[l]} := \beta^{[l]} - \alpha d\beta^{[l]}$

$\gamma^{[l]} := \gamma^{[l]} - \alpha d\gamma^{[l]}$